

6

Interprocess Communications

6.1 Introduction

A complex programming environment often uses multiple cooperating processes to perform related operations. These processes must communicate with each other and share resources and information. The kernel must provide mechanisms that make this possible. These mechanisms are collectively referred to as *interprocess communications*, or IPC. This chapter describes the IPC facilities in major UNIX variants.

Interprocess interactions have several distinct purposes:

- **Data transfer** — One process may wish to send data to another process. The amount of data sent may vary from one byte to several megabytes.
- **Sharing data** — Multiple processes may wish to operate on shared data, such that if a process modifies the data, that change will be immediately visible to other processes sharing it.
- **Event notification** — A process may wish to notify another process or set of processes that some event has occurred. For instance, when a process terminates, it may need to inform its parent process. The receiver may be notified asynchronously, in which case its normal processing is interrupted. Alternatively, the receiver may choose to wait for the notification.
- **Resource sharing** — Although the kernel provides default semantics for resource allocation, they are not suitable for all applications. A set of cooperating processes may wish to define their own protocol for accessing specific resources. Such rules are usually imple-

mented by a locking and synchronization scheme, which must be built on top of the basic set of primitives provided by the kernel.

- **Process control** — A process such as a debugger may wish to assume complete control over the execution of another (target) process. The controlling process may wish to intercept all traps and exceptions intended for the target and be notified of any change in the target's state.

UNIX provides several different IPC mechanisms. This chapter first describes a core set of facilities found in all versions of UNIX, namely signals, pipes, and process tracing. It then examines the primitives collectively described as *System V IPC*. Finally, it looks at message-based IPC in Mach, which provides a rich set of facilities from a single, unified framework.

6.2 Universal IPC Facilities

When the UNIX system was first released externally [Thom 78], it provided three facilities that could be used for interprocess communications—signals, pipes, and process tracing.¹ These are the only IPC mechanisms common to all UNIX variants. Signals and pipes are described in greater detail elsewhere in this book; in this section, we discuss how they are used for interprocess communications.

6.2.1 Signals

Signals serve primarily to notify a process of asynchronous events. Originally intended for handling errors, they are also used as primitive IPC mechanisms. Modern UNIX versions recognize 31 or more different signals. Most have predefined meanings, but at least two, SIGUSR1 and SIGUSR2, are available for applications to use as they please. A process may explicitly send a signal to another process or processes using the *kill* or *killpg* system calls. Additionally, the kernel generates signals internally in response to various events. For instance, typing control-C at the terminal sends a SIGINT signal to the foreground process.

Each signal has a default action, which is typically to terminate the process. A process may specify an alternative response to any signal by providing a signal handler function. When the signal occurs, the kernel interrupts the process, which responds to the signal by running the handler. When the handler completes, the process may resume normal processing.

This way processes are notified of, and respond to, asynchronous events. Signals can also be used for synchronization. A process may use the *sigpause* call to wait for a signal to arrive. In early UNIX releases, many applications developed resource sharing and locking protocols based entirely on signals.

The original intent was to use signals primarily for handling errors; for example, the kernel translates hardware exceptions, such as *division by zero* or *invalid instruction*, into signals. If the process does not have an error handler for that exception, the kernel terminates the process.

¹ The earliest UNIX systems in Bell Telephone Laboratories did not have these features. For instance, pipes were developed by Doug McIlroy and Ken Thompson and made available in Version 3 UNIX in 1973 [Salu 94].

As an IPC mechanism, signals have several limitations: Signals are expensive. The sender must make a system call; the kernel must interrupt the receiver and extensively manipulate its stack, so as to invoke the handler and later resume the interrupted code. Moreover, they have a very limited bandwidth—because only 31 different signals exist (in SVR4 or 4.3BSD; some variants such as AIX provide more signals), a signal can convey only a limited amount of information. It is not possible to send additional information or arguments with user-generated signals.² Signals are useful for event notification, but are inefficient for more complicated interactions.

Signals are discussed in detail in Chapter 4.

6.2.2 Pipes

In traditional implementations, a pipe is a unidirectional, first-in first-out, unstructured data stream of fixed maximum size.³ Writers add data to the end of the pipe; readers retrieve data from the front of the pipe. Once read, the data is removed from the pipe and is unavailable to other readers. Pipes provide a simple flow-control mechanism. A process attempting to read from an empty pipe blocks until more data is written to the pipe. Likewise, a process trying to write to a full pipe blocks until another process reads (and thus removes) data from the pipe.

The *pipe* system call creates a pipe and returns two file descriptors—one for reading and one for writing. These descriptors are inherited by child processes, which thus share access to the file. This way, each pipe can have several readers and writers (Figure 6-1). A given process may be a reader or writer, or both. Normally, however, the pipe is shared between two processes, each owning one end. I/O to the pipe is much like I/O to a file and is performed through *read* and *write* system calls to the pipe's descriptors. A process is often unaware that the file it is reading or writing is in fact a pipe.

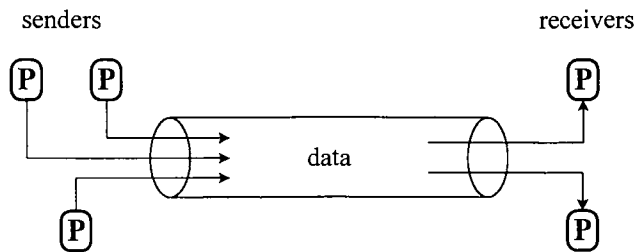


Figure 6-1. Data flow through a pipe.

² Signals generated by the kernel in response to hardware exceptions return additional information via the `siginfo` structure passed to the handler.

³ Traditional UNIX systems such as SVR2 implement pipes in the file system and use the direct block address fields in the inode (see Section 9.2.2) to locate data blocks of the pipe. This limits the pipe size to ten blocks. Newer UNIX systems retain this limit, even though they implement pipes differently.

Typical applications such as the *shell* manipulate the descriptors so that a pipe has exactly one reader and one writer, thus using it for one-way flow of data. The most common use of pipes is to let the output of one program become the input for another. Users typically join two programs by a pipe using the shell's pipe operator (`|`).

From the IPC perspective, pipes provide an efficient way of transferring data from one process to another. They have, however, some important limitations:

- Since reading data removes it from the pipe, a pipe cannot be used to broadcast data to multiple receivers.
- Data in a pipe is treated as a byte-stream and has no knowledge of message boundaries. If a writer sends several objects of different length through the pipe, the reader cannot determine how many objects have been sent, or where one object ends and the next begins.⁴
- If there are multiple readers on a pipe, a writer cannot direct data to a specific reader. Likewise, if there are multiple writers, there is no way to determine which of them sent the data.⁵

There are several ways to implement pipes. The traditional approach (in SVR2, for instance) is to use the file system mechanisms and associate an *inode* and a *file table entry* with each pipe. Many BSD-based variants use sockets to implement a pipe. SVR4 provides bidirectional, STREAMS-based pipes, described in the following section.

A related facility available in System V UNIX, and in many commercial variants, is the FIFO (first-in, first-out) file, also called a named pipe. These differ from pipes mainly in the way they are created and accessed. A user creates a FIFO file by calling *mknod*, passing it a filename and a creation mode. The mode field includes a file type of `S_IFIFO` and the usual access permissions. Thereafter, any process that has the appropriate permissions can open the FIFO and read or write to it. The semantics of reading and writing a FIFO are very similar to those of a pipe, and are further described in Section 8.4.2. The FIFO continues to exist until explicitly unlinked, even if no readers or writers are active.

FIFOs offer some important advantages over pipes. They may be accessed by unrelated processes. They are persistent, and hence are useful for data that must outlive the active users. They have a name in the file system name space. FIFOs also have some important drawbacks. They must be explicitly deleted when not in use. They are less secure than pipes, since any process with the right privileges can access them. Pipes are easier to set up and consume fewer resources.

6.2.3 SVR4 Pipes

SVR4 uses STREAMS (see Chapter 17) as its basic framework for networking and also to implement pipes and FIFOs. Doing so allows it to provide several new and useful features for pipes.⁶ This section describes the new functionality; Section 17.9 covers the implementation details.

SVR4 pipes are bidirectional. The *pipe* system call returns two descriptors as before, but both are open for reading and writing. The syntax (same as for traditional UNIX) is

⁴ Cooperating applications could agree on a protocol to store packet boundary information in each object.

⁵ Again, cooperating applications could agree on a protocol for tagging the data source for each object.

⁶ This new functionality applies only to pipes. SVR4 FIFOs behave much like traditional ones.

```
status = pipe (int fildes[2]);
```

In SVR4, this call creates two independent, first-in first-out, I/O channels that are represented by the two descriptors. Data written to `fildes[1]` can be read from `fildes[0]`, and data written to `fildes[0]` can be read from `fildes[1]`. This is very useful because many applications require two-way communication, and used two separate pipes in pre-SVR4 implementations.

SVR4 also allows a process to attach any STREAMS file descriptor to an object in the file system [Pres 90]. An application can create a pipe using *pipe*, and then bind either of its descriptors to a file name by calling

```
status = fattach (int fildes, char *path);
```

where `path` is the pathname of a file system object owned by, and writable by, the caller.⁷ This object can be an ordinary file, directory, or other special file. It cannot be an active mount point (cannot have a file system mounted on it) or an object in a remote file system, and also cannot be attached to another STREAMS file descriptor. It is possible to attach a descriptor to several paths, thereby associating multiple names with it.

Once attached, all subsequent operations on `path` will operate on the STREAMS file until the descriptor is detached from `path` through *fdetach*. Using this facility, a process can create a pipe and then allow unrelated processes to access it.

Finally, users can push STREAMS modules onto a pipe or FIFO. These modules intercept the data flowing through the stream and process it in some way. Because the modules execute inside the kernel, they can provide functionality not possible with a user-level application. Nonprivileged users cannot add modules to the system, but can push modules onto streams they have opened.

6.2.4 Process Tracing

The *ptrace* system call provides a basic set of facilities for process tracing. It is primarily used by debuggers such as *sdb* and *dbx*. Using *ptrace*, a process can control the execution of a child process. Multiple children can be controlled with *ptrace*, though that feature is seldom used. The syntax for *ptrace* is

```
ptrace (cmd, pid, addr, data);
```

where `pid` is the process ID of the target process, `addr` refers to a location in the target's address space, and interpretation of the `data` argument depends on `cmd`. The `cmd` argument allows the parent to perform the following operations:

- Read or write a word in the child's address space.
- Read or write a word in the child's u area.
- Read or write to the child's general-purpose registers.

⁷ Or else the caller must be a privileged process.

- Intercept specific signals. When an intercepted signal is generated for the child, the kernel will suspend the child and notify the parent of the event.
- Set or delete *watchpoints* in the child's address space.
- Resume the execution of a stopped child.
- Single-step the child—resume its execution, but arrange for it to stop again after executing one instruction.
- Terminate the child.

One command (`cmd == 0`) is reserved for the child. The child uses this command to inform the kernel that it (the child) will be traced by its parent. The kernel sets the child's *traced* flag (in its *proc* structure), which affects how the child responds to signals. If a signal is generated to a traced process, the kernel suspends the process and notifies its parent via a `SIGCHLD` signal, instead of invoking the signal handler. This allows the parent to intercept the signal and act appropriately. The *traced* flag also changes the behavior of the *exec* system call. When the child invokes a new program, *exec* generates a `SIGTRAP` signal to the child before returning to user mode. Again, this allows the parent to gain control of the child before the child begins to run.

The parent typically creates the child, which invokes *ptrace* to allow the parent to control it. The parent then uses the *wait* system call to wait for an event that changes the child's state. When the event occurs, the kernel wakes up the parent. The return value of *wait* indicates that the child has stopped rather than terminated and supplies information about the event that caused the child to stop. The parent then controls the child by one or more *ptrace* commands.

Although *ptrace* has allowed the development of many debuggers, it has several important drawbacks and limitations:

- A process can only control the execution of its direct children. If the traced process *forks*, the debugger cannot control the new process or its descendants.
- *ptrace* is extremely inefficient, requiring several context switches to transfer a single word from the child to the parent. These context switches are necessary because the debugger does not have direct access to the child's address space.
- A debugger cannot trace a process that is already running, because the child first needs to call *ptrace* to inform the kernel that it is willing to be traced.
- Tracing a *setuid* program raises a security problem if such a program subsequently calls *exec*. A crafty user could use the debugger to modify the target's address space, so that *exec* invokes the shell instead of the program it was asked to run. As a result, the user obtains a shell with superuser privileges. To avoid this problem, UNIX either disables tracing of *setuid* programs or inhibits the *setuid* and *setgid* actions on subsequent *exec* calls.

For a long time, *ptrace* was the only tool for debugging programs. Modern UNIX systems such as SVR4 and Solaris provide much more efficient debugging facilities using the */proc* file system [Faul 91], described in Section 9.11.2. It is free from the limitations of *ptrace* and provides additional capabilities such as allowing a process to debug unrelated processes, or allowing a debugger to attach to a running process. Many debuggers have been rewritten to use */proc* instead of *ptrace*.

6.3 System V IPC

The facilities described in the previous section do not satisfy the IPC requirements of many applications. A major advancement came with System V UNIX, which provided three mechanisms—*semaphores*, *message queues*, and *shared memory*—that have collectively come to be known as System V IPC [Bach 86]. They were originally designed to support the needs of transaction-processing applications. Subsequently, they have been implemented by most UNIX vendors, including those making BSD-based systems. This section describes the functionality provided by these mechanisms and how they are implemented in UNIX.

6.3.1 Common Elements

The three mechanisms are very similar in their interface to the programmer, as well as their implementation. In describing these common features, we use the term *IPC resource* (or simply, a resource) to refer to a single semaphore set, message queue, or shared memory region. Each instance of an IPC resource has the following attributes:

- **Key** — a user-supplied integer that identifies this instance of the resource.
- **Creator** — user and group IDs of the process that created the resource.
- **Owner** — user and group IDs of the owner of the resource. When the resource is created, the creator of the resource is also its owner. Subsequently, a process that has permission to change the owner may specify a new owner. The creator, current owner, and superuser processes have this permission.
- **Permissions** — file system-type read/write/execute permissions for owner, group, and others.

A process acquires a resource by making a *shmget*, *semget*, or *msgget* system call, passing to it the key, certain flags, and other arguments that depend on the specific mechanism. The permitted flags are `IPC_CREAT` and `IPC_EXCL`. `IPC_CREAT` asks the kernel to create the resource if it does not exist. `IPC_EXCL` is used in conjunction with `IPC_CREAT` and asks the kernel to return an error if the resource already exists. If no flags are specified, the kernel searches for an existing resource with the same key.⁸ If found, and if the caller has permission to access it, the kernel returns a *resource ID* that may be used to locate the resource quickly for further operations.

Each mechanism has a control system call (*shmctl*, *semctl*, or *msgctl*) that provides several commands. The commands include `IPC_STAT` and `IPC_SET` to obtain and set status information (the specifics depend on the mechanism), and `IPC_RMID` to deallocate the resource. Semaphores provide additional control commands to obtain and set values of individual semaphores in the set.

Each IPC resource must be explicitly deallocated by the `IPC_RMID` command. Otherwise, the kernel considers it to be active even if all processes that were using it have terminated. This property can be quite useful. For instance, a process may write data to a shared memory region or message

⁸ If the key is the special value `IPC_PRIVATE`, the kernel creates a new resource. This resource cannot be accessed through other *get* calls (since the kernel will generate a new resource each time), and hence the caller has exclusive ownership of it. The owner can share the resource with its children, who inherit it through the *fork* system call.

queue and then exit; at a later time, another process can retrieve this data. The IPC resource can persist and be usable beyond the lifetime of the processes accessing it.

The drawback, however, is that the kernel cannot determine if a resource has deliberately been left active for use by future processes, or if it has been abandoned accidentally, perhaps because the process that would have freed it terminated abnormally before doing so. As a result, the kernel must retain the resource indefinitely. If this happens often, the system could run out of that resource. At the very least, the resource ties up memory that could be better used.

Only the creator, current owner, or a superuser process can issue the `IPC_RMID` command. Removing a resource affects all processes that are currently accessing it, and the kernel must ensure that these processes handle this event gracefully and consistently. The specifics of this issue differ for each IPC mechanism and are discussed in the following sections.

To implement this interface, each type of resource has its own fixed-size resource table. The size of this table is configurable and limits the total number of instances of that IPC mechanism that can simultaneously be active in the system. The resource table entry comprises a common `ipc_perm` structure and a part specific to the type of resource. The `ipc_perm` structure contains the common attributes of the resources (the key, creator and owner IDs, and permissions), as well as a sequence number, which is a counter that is increased each time the entry is reused.

When a user allocates an IPC resource, the kernel returns the resource ID, which it computes by the formula

```
id = seq * table_size + index;
```

where `seq` is the sequence number of this resource, `table_size` is the size of the resource table, and `index` is the index of the resource in the table. This ensures that a new `id` is generated if a table element is reused, since `seq` is incremented. This prevents processes from accessing a resource using a stale `id`.

***Note:** To **increment** a variable means to increase its value by one; to **decrement** it means to decrease its value by one. These terms are derived from the increment (++) and decrement (--) operators of C.*

The user passes the `id` as an argument to subsequent system calls on that resource. The kernel translates the `id` to locate the resource in the table using the formula

```
index = id % table_size;
```

6.3.2 Semaphores

Semaphores [Dijk 65] are integer-valued objects that support two atomic operations—`P()` and `V()`.⁹ The `P()` operation decrements the value of the semaphore and blocks if its new value is less than

⁹ The names `P()` and `V()` are derived from the Dutch words for these operations.

zero. The `V()` operation increments its value; if the resulting value becomes greater than or equal to zero, `V()` wakes up a waiting thread or process (if any). The operations are atomic in nature.

Semaphores may be used to implement several synchronization protocols. For example, consider the problem of managing a counted resource, that is, a resource with a fixed number of instances. Processes try to acquire an instance of the resource, and release it when they finish using it. This resource can be represented by a semaphore that is initialized to the number of instances. The `P()` operation is used while trying to acquire the resource; it will decrement the semaphore each time it succeeds. When the value reaches zero (no free resources), further `P()` operations will block. Releasing a resource results in a `V()` operation, which increments the value of the semaphore, causing blocked processes to awaken.

In many UNIX systems, the kernel uses semaphores internally to synchronize its operations. It is also desirable to provide the same facility for applications. System V provides a generalized version of semaphores. The `semget` system call creates or obtains an array of semaphores (there is a configurable upper bound on the array size). Its syntax is

```
semid = semget (key, count, flag);
```

where `key` is a 32-bit value supplied by the caller. `semget` returns an array of count semaphores associated with `key`. If no semaphore set is associated with `key`, the call fails unless the caller has supplied the `IPC_CREAT` flag, which creates a new semaphore set. If the `IPC_EXCL` flag was also provided, `semget` returns an error if a semaphore set already exists for that `key`. The `semid` value is used in subsequent semaphore operations to identify this semaphore array.

The `semop` system call is used to perform operations on the individual semaphores in this array. Its syntax is

```
status = semop (semid, sops, nsops);
```

where `sops` is a pointer to an `nsops`-element array of `sembuf` structures. Each `sembuf`, as described below, represents one operation on a single semaphore in the set:

```
struct sembuf {
    unsigned short sem_num;
    short sem_op;
    short sem_flg;
};
```

`sem_num` identifies one semaphore from the array and `sem_op` specifies the action to perform on it. The value of `sem_op` is interpreted as follows:

<code>sem_op > 0</code>	Add <code>sem_op</code> to the semaphore's current value. This may result in waking up processes that are waiting for the value to increase.
<code>sem_op = 0</code>	Block until the semaphore's value becomes zero.

`sem_op < 0` Block until the semaphore's value becomes greater than or equal to the absolute value of `sem_op`, then subtract `sem_op` from that value. If the semaphore's value is already greater than the absolute value of `sem_op`, the caller does not block.

Thus, a single *semop* call can specify several individual operations, and the kernel guarantees that either all or none of the operations will complete. Moreover, the kernel guarantees that no other *semop* call on this array will begin until this one completes or blocks. If a *semop* call must block after completing some of its component operations, the kernel *rewinds* the operation to the beginning (undoes all modifications) to ensure atomicity of the entire call.

The `sem_flg` argument can supply two flags to the call. The `IPC_NOWAIT` flag asks the kernel to return an error instead of blocking. Also, a deadlock may occur if a process holding a semaphore exits prematurely without releasing it. Other processes waiting to acquire it may block forever in the `P()` operation. To avoid this, a `SEM_UNDO` flag can be passed to *semop*. If so, the kernel remembers the operation, and automatically rolls it back if the process exits.

Finally, semaphores must be explicitly removed by the `IPC_RMID` command of the *semctl* call. Otherwise, the kernel retains them even they are no longer being used by any process. This allows semaphores to be used in situations that span process lifetimes, but can tie up resources if applications exit without destroying the semaphores.

When a process issues the `IPC_RMID` command, the kernel frees up the semaphore in the resource table. The kernel also wakes up any processes that have blocked on some semaphore operation; these processes return an `EIDRM` status from the *semop* call. Once the semaphore is removed, processes can no longer access it (whether using the key or the `semid`).

Implementation Details

The kernel translates the `semid` to obtain the semaphore resource table entry, which is described by the following data structure:

```
struct semid_ds {
    struct ipc_perm sem_perm;    /* see Section 6.3.1 */
    struct sem* sem_base;       /* pointer to array of semaphores in set */
    ushort sem_nsems;           /* number of semaphores in set */
    time_t sem_otime;           /* last operation time */
    time_t sem_ctime;           /* last change time */
    ...
};
```

For each semaphore in the set, the kernel maintains its value and synchronization information in the following structure:

```

struct sem {
    ushort semval; /* current value */
    pid_t sempid; /* pid of process that invoked the last operation */
    ushort semncnt; /* num of procs waiting for semval to increase */
    ushort semzcnt; /* num of procs waiting for semval to equal 0 */
};

```

Finally, the kernel maintains an *undo list* for each process that has invoked semaphore operations with the SEM_UNDO flag. This list contains a record of each operation that must be rolled back. When a process exits, the kernel checks if it has an undo list; if so, the kernel traverses the list and reverses all the operations.

Discussion

Semaphores allow the development of complex synchronization facilities for use by cooperating processes. On early UNIX systems that did not support semaphores, applications requiring synchronization sought and used other atomic operations in UNIX. One alternative is the *link* system call, which fails if the new link already exists. If two processes try the same *link* operation at the same time, only one of them will succeed. It is, however, expensive and senseless to use file system operations such as *link* merely for interprocess synchronization, and semaphores fill a major need of application programmers.

The major problems with semaphores involve race conditions and deadlock avoidance. Simple semaphores (as opposed to semaphore arrays) can easily cause a deadlock if processes must acquire multiple semaphores. For example, in Figure 6-2, process A holds semaphore S1 and tries to acquire semaphore S2, while process B holds S2 and tries to acquire S1. Neither process can progress. Although this simple case is easy to detect and avoid, a deadlock can occur in an arbitrarily complex scenario involving several semaphores and processes.

It is impractical to have deadlock detection and avoidance code in the kernel. Moreover, there are no general, bounded algorithms that apply to all possible situations. The kernel thus leaves all deadlock detection to the applications. By providing semaphore sets with compound atomic operations, the kernel supplies mechanisms to handle multiple semaphores intelligently. Applications can choose from several well-known deadlock avoidance techniques, some of which are discussed in Section 7.10.1.

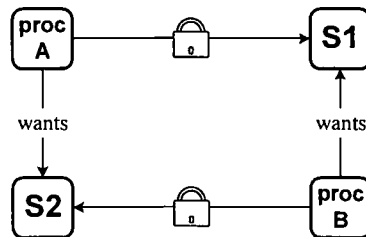


Figure 6-2. Semaphores can cause deadlocks.

One major problem in the System V semaphore implementation is that the allocation and initialization of the semaphores are not atomic. The user calls *semget* to allocate a semaphore set, followed by *semctl* to initialize it. This can lead to race conditions that must be prevented at the application level [Stev 90].

Finally, the need to explicitly delete the resource through *IPC_RMID* is a common problem with all IPC mechanisms. Although it allows the lifetime of the resource to exceed that of its creator, it creates a garbage collection problem if processes exit without destroying their resources.

6.3.3 Message Queues

A message queue is a header pointing at a linked list of messages. Each message contains a 32-bit *type* value, followed by a *data* area. A process creates or obtains a message queue using the *msgget* system call, which has the syntax

```
msgqid = msgget (key, flag);
```

The semantics of the call are similar to those of *semget*. The key is a user-chosen integer. The *IPC_CREAT* flag is required to create a new message queue, and *IPC_EXCL* causes the call to fail if a queue already exists for that key. The *msgqid* value is used in further calls to access the queue.

The user places messages on the queue by calling

```
msgsnd (msgqid, msgp, count, flag);
```

where *msgp* points to the message buffer (containing a *type* field followed by data area), and *count* is the total number of bytes in the message (including the *type* field). The *IPC_NOWAIT* flag can be used to return an error if the message cannot be sent without blocking (if the queue is full, for example—the queue has a configurable limit on the total amount of data it can hold).

Figure 6-3 describes the operation of a message queue. Each queue has an entry in the message queue resource table, and is represented by the following structure:

```
struct msqid_ds {
    struct ipc_perm msg_perm;    /* described in Section 6.3.1 */
    struct msg* msg_first;      /* first message on queue */
    struct msg* msg_last;      /* last message on queue */
    ushort msg_cbytes;          /* current byte count on queue */
    ushort msg_qbytes;          /* max bytes allowed on queue */
    ushort msg_qnum;            /* number of messages currently on queue */
    ...
};
```

Messages are maintained in the queue in order of arrival. They are removed from the queue (in first-in, first-out order) when read by a process, using the call

```
count = msgrcv (msgqid, msgp, maxcnt, msgtype, flag);
```

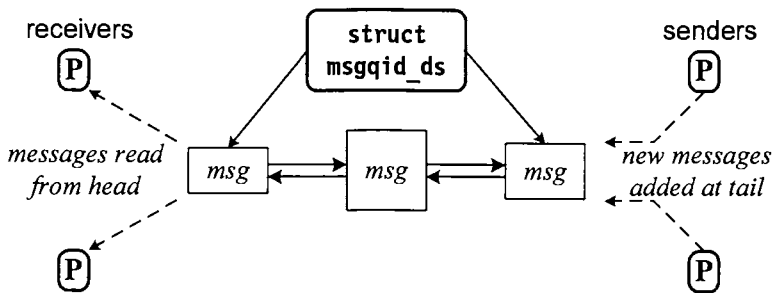


Figure 6-3. Using a message queue.

Here `msgp` points to a buffer into which the incoming message will be placed, and `maxcnt` limits the amount of data that can be read. If the incoming message is larger than `maxcnt` bytes, it will be truncated. The user must ensure that the buffer pointed to by `msgp` is large enough to hold `maxcnt` bytes. The return value specifies the number of bytes successfully read.

If `msgtype` equals zero, `msgrcv` returns the first message on the queue. If `msgtype` is greater than zero, `msgrcv` returns the first message of type `msgtype`. If `msgtype` is less than zero, `msgrcv` returns the first message of the lowest type that is less than or equal to the absolute value of `msgtype`. Again, the `IPC_NOWAIT` flag causes the call to return immediately if an appropriate message is not on the queue.

Once read, the message is removed from the queue and cannot be read by other processes. Likewise, if a message is truncated because the receiving buffer is too small, the truncated part is lost forever, and no indication is given to the receiver.

A process must explicitly delete a message queue by calling `msgctl` with the `IPC_RMID` command. When this happens, the kernel frees the message queue and deletes all messages on it. If any processes are blocked waiting to read or write to the queue, the kernel awakens them, and they return from the call with an `EIDRM` (message ID has been removed) status.

Discussion

Message queues and pipes provide similar services, but message queues are more versatile and address several limitations of pipes. Message queues transmit data as discrete messages rather than as an unformatted byte-stream. This allows data to be processed more intelligently. The message *type* field can be used in various ways. For instance, it can associate priorities with messages, so that a receiver can check for urgent messages before reading nonurgent ones. In situations where a message queue is shared by multiple processes, the *type* field can be used to designate a recipient.

Message queues are effective for transferring small amounts of data, but become very expensive for large transfers. When one process sends a message, the kernel copies the message into an internal buffer. When another process retrieves this message, the kernel copies the data to the receiver's address space. Thus each message transfer involves two data copy operations, resulting in poor performance. Later in this chapter, we describe how Mach IPC allows efficient transfer of large amounts of data.

Another limitation of message queues is that they cannot specify a receiver. Any process with the appropriate permissions can retrieve messages from the queue. Although, as mentioned earlier, cooperating processes can agree on a protocol to specify recipients, the kernel does not assist with this. Finally, message queues do not supply a *broadcast* mechanism, whereby a process can send a single message to several receivers.

The STREAMS framework, available in most modern UNIX systems, provides a rich set of facilities for message passing. It provides more functionality than, and is more efficient than, message queues, rendering the latter almost obsolete. One feature that is available in message queues but not in STREAMS is the ability to retrieve messages selectively, based on their types. This may be useful for certain applications. Most applications, however, find STREAMS more useful, and message queues have been retained in modern UNIX variants primarily for backward compatibility. Chapter 17 describes STREAMS in detail.

6.3.4 Shared Memory

A shared memory region is a portion of physical memory that is shared by multiple processes. Processes may attach this region to any suitable virtual address range in their address space. This address range may differ for each process (Figure 6-4). Once attached, the region may be accessed like any other memory location without requiring system calls to read or write data to it. *Hence, shared memory provides the fastest mechanism for processes to share data.* If a process writes to a shared memory location, the new contents of that location are immediately visible to all processes sharing the region.¹⁰

A process initially obtains or creates a shared memory region by calling

```
shmid = shmget (key, size, flag);
```

where *size* is the size of this region, and the other parameters and flags are the same as for *semget*

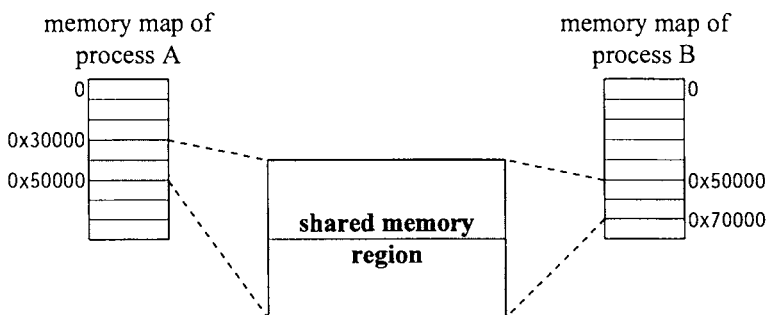


Figure 6-4. Attaching a shared memory region.

¹⁰ On a multiprocessor, additional operations are required to ensure cache consistency. Section 15.13 discusses some of these issues.

or *msgget*. The process then attaches the region to a virtual address, using

```
addr = shmat (shmid, shmaddr, shmflag);
```

The *shmaddr* argument suggests an address to which the region may be attached. The *shmflag* argument can specify the *SHM_RND* flag, which asks the kernel to round *shmaddr* down by an appropriate alignment factor. If *shmaddr* is zero, the kernel is free to choose any address. The *SHM_RDONLY* flag specifies that the region should be attached as read-only. The *shmat* call returns the actual address to which the region was attached.

A process can detach a shared memory region from its address space by calling

```
shmdt (shmaddr);
```

To destroy the region completely, a process must use the *IPC_RMID* command of the *shmctl* system call. This marks the region for deletion, and the region will be destroyed when all processes detach it from their address space. The kernel maintains a count of processes attached to each region. Once a region has been marked for deletion, new processes cannot attach to it. If the region is not explicitly deleted, the kernel will retain it even if no process is attached to it. This may be desirable for some applications: a process could leave some data in a shared memory region and terminate; at a later time, another cooperating process could attach to this region using the same key and retrieve the data.

The implementation of shared memory depends heavily on the virtual memory architecture of the operating system. Some variants use a single *page table* to map the shared memory region and share the table with all processes attached to it. Others have separate, per-process address translation maps for the region. In such a model, if one process performs an action that changes the address mapping of a shared page, the change must be applied to all mappings for that page. SVR4, whose memory management is described in Chapter 14, uses an *anon_map* structure to locate the pages of the shared memory region. Its shared memory resource table contains entries represented by the following structure:

```
struct shmid_ds {
    struct ipc_perm shm_perm;    /* described in Section 6.3.1 */
    int shm_segsz;               /* segment size in bytes */
    struct anon_map *shm_amp;    /* pointer to memory mgmt info */
    ushort shm_nattch;           /* number of current attaches */
    ...
};
```

Shared memory provides a very fast and versatile mechanism that allows a large amount of data to be shared without copying or using system calls. Its main limitation is that it does not provide synchronization. If two processes attempt to modify the same shared memory region, the kernel will not serialize the operations, and the data written may be arbitrarily intermingled. Processes sharing a shared memory region must devise their own synchronization protocol, and they usually do so using primitives such as semaphores. These primitives involve one or more system calls that impose an overhead on shared memory performance.

Most modern UNIX variants (including SVR4) also provide the *mmap* system call, which maps a file (or part of a file) into the address space of the caller. Processes can use *mmap* for IPC by mapping the same file into their address space (in the *MAP_SHARED* mode). The effect is similar to a shared memory region that is initialized to the contents of the file. If a process modifies a mapped file, the change is immediately visible to all processes that have mapped that file; the kernel will also update the file on disk. One advantage of *mmap* is that it uses the file system name space instead of keys. Unlike shared memory, whose pages are backed by swap space (see Section 14.7.6), *mmap*'ed pages are backed by the file to which they are mapped. Section 14.2 describes *mmap* in detail.

6.3.5 Discussion

There are several similarities between the IPC mechanisms and the file system. The resource ID is like a file descriptor. The *get* calls resemble *open*, *IPC_RMID* resembles *unlink*, and the *send* and *receive* calls resemble *read* and *write*. The *shmdt* call provides *close*-like functionality for shared memory. For message queues and semaphores, however, there is no equivalent of the *close* system call, which might be desirable for removing resources cleanly. As a result, processes using a message queue or semaphore may suddenly find that the resource no longer exists.

In contrast, the keys associated with a resource form a name space that is distinct from the file system name space.¹¹ Each mechanism has its own name space, and the key uniquely identifies a resource within it. Because the key is a simple, user-chosen integer, it is useful only on a single machine and is unsuitable for a distributed environment. Also, it is difficult for unrelated processes to choose and agree upon an integer-valued key and avoid conflicts with keys used by other applications. Hence, UNIX provides a library routine called *ftok* (described in the *stdipc(3C)* manual page) to generate a key that is based on a file name and an integer. Its syntax is

```
key = ftok (char *pathname, int ndx);
```

The *ftok* routine generates a key value, usually based on *ndx* and the inode number of the file. An application can choose a unique file name more easily than a unique integer value (for instance, it can use the pathname of its own executable file), and hence reduce the likelihood of key conflicts. The *ndx* parameter allows greater flexibility and can be used, for example, to specify a project ID known to all cooperating processes.

Security is a problem, because the resource IDs are actually indexes into a global resource table. An unauthorized process can access a resource simply by guessing the ID. It can thus read or write messages or shared memory, or tamper with semaphores used by other processes. The permissions associated with each resource offer some protection, but many applications must share the resources with processes belonging to different users, and hence cannot use very restrictive permissions. Using the sequence number as a component of the resource ID provides a little more protection, since there are more IDs to guess from, but still poses a serious problem for applications that are concerned about security.

¹¹ Several operating systems, such as Windows/NT and OS/2, use pathnames to name shared memory objects.

Much of the functionality provided by System V IPC can be duplicated by other file system facilities such as file locking or pipes. However, the IPC facilities are much more versatile and flexible, and offer better performance than their file system counterparts.

6.4 Mach IPC

The remainder of this chapter discusses Mach's message-based IPC facility. In Mach, IPC is the central and most important kernel component. Instead of the operating system supporting IPC mechanisms, Mach provides an IPC facility that supports much of the operating system. There were several important goals in the design of Mach IPC:

- Message passing must be the fundamental communication mechanism.
- The amount of data in a single message may range from a few bytes to an entire address space (typically up to four gigabytes). The kernel should enable large transfers without unnecessary data copying.
- The kernel should provide secure communications and allow only authorized threads to send and receive messages.
- Communication and memory management are tightly coupled. The IPC subsystem uses the copy-on-write mechanisms of the memory subsystem to efficiently transfer large amounts of data. Conversely, the memory subsystem uses IPC to communicate with user-level memory managers (known as "external pagers").
- Mach IPC must support communication between user tasks, and also between the user and the kernel. In Mach, a thread makes a system call by sending a message to the kernel, and the kernel returns the result in a reply message.
- The IPC mechanism should be suitable for applications based on the client-server model. Mach uses user-level server programs to perform many services (such as file system and memory management) that are traditionally handled by the operating system kernel. These servers use Mach IPC to handle requests for service.
- The interface should be transparently extensible to a distributed environment. The user should not need to know whether he is sending a message to a local receiver or to a remote node.

Mach IPC has evolved steadily over various releases. Sections 6.4 to 6.9 discuss the IPC facility in Mach 2.5, which is the most popular Mach release and the basis for operating systems such as OSF/1 and Digital UNIX. Mach 3.0 enhances the IPC mechanisms in several respects; these features are discussed in Section 6.10.

This chapter makes several references to Mach tasks and threads. Section 3.7.1 discusses these abstractions in detail. In brief, a *task* is a collection of resources, including an address space in which one or more threads execute. A *thread* is a dynamic entity that represents an independent program counter and stack—thus a logical control sequence—in a program. A UNIX process is equivalent to a task containing a single thread. All threads in a task share the resources of that task.

6.4.1 Basic Concepts

The two fundamental IPC abstractions in Mach are messages and ports [Rash 86]. A *message* is a collection of typed data. A *port* is a protected queue of messages. A message can be sent only to a port, not to a task or thread. Mach associates *send rights* and *receive rights* with each port. These rights are owned by tasks. A send right allows a task to send messages to the port; a receive right allows it to receive messages sent to the port. Several tasks may own send rights to a single port, but only one task, the *owner* of the port, holds the receive rights.¹² Thus a port allows many-to-one communication, as illustrated in Figure 6-5.

A message may be simple or complex. A simple message contains ordinary data that is not interpreted by the kernel. A complex message may contain ordinary data, out-of-line memory (data that is passed by reference, using copy-on-write semantics), and send or receive rights to various ports. The kernel interprets the information in a complex message and transforms it to a form meaningful to the receiver.

Each port has a reference count that monitors the number of rights to it. Each such right (also known as a *capability*) represents one name of that port. The names are integers, and the name space is local to each task. Thus two tasks may have different names for the same port (Figure 6-6). Conversely, the same port name may refer to different ports in different tasks.

Ports also represent kernel objects. Hence each object, such as a task, thread, or processor, is represented by a port. Rights to these ports represent object references and allow the holder to perform operations on that object. The kernel holds the receive rights to such ports.

Each port has a finite-size message queue. The size of this queue provides a simple flow-control mechanism. Senders are blocked when the queue is full, and receivers when the queue is empty.

Each task and thread has a default set of ports. For instance, each task has send rights to a *task_self* port that represents itself (the kernel has receive rights to this port) and receive rights to a *task_notify* port (to which the kernel has send rights). Tasks also have send rights to a *bootstrap* port that provides access to a name server. Each thread has send rights to a *thread_self* port, and receive

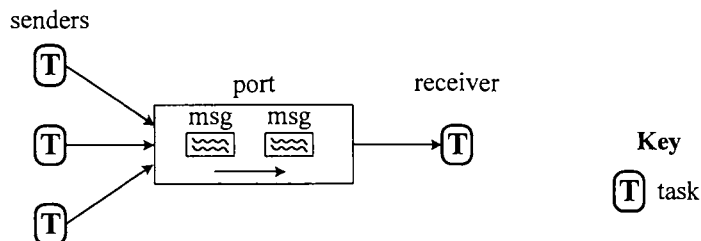


Figure 6-5. Communicating via a Mach port.

¹² Earlier versions of Mach had separate ownership and receive rights. Mach 2.5 and newer releases replace the ownership rights with the *backup port* facility, which is described in Section 6.8.2.

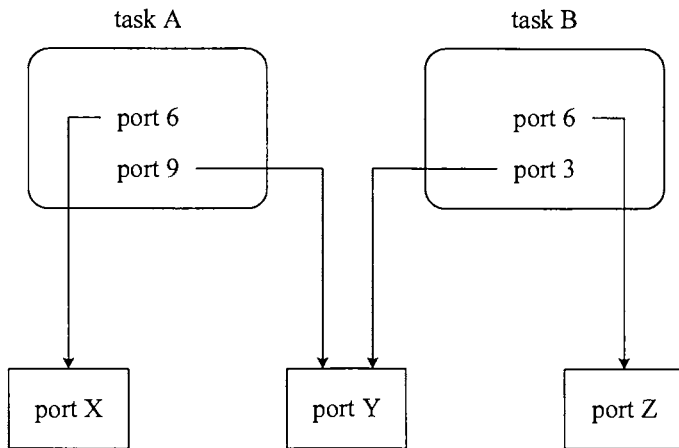


Figure 6-6. Local names for ports.

rights to a *reply* port, used to receive replies from system calls and remote procedure calls to other tasks. There is also an *exception* port associated with each task and each thread. The rights to the per-thread ports are owned by the task in which the thread runs; hence these ports can be accessed by all threads within the task.

Tasks also inherit other port rights from their parent. Each task has a list of *registered ports*. These allow the task to access various system-wide services. These ports are inherited by new tasks during task creation.

6.5 Messages

Mach is a message-passing kernel, and most system services are accessed by exchanging messages. Mach IPC provides communication between user tasks, between users and the kernel, and between different kernel subsystems. A user-level program called the *netmsgserver* transparently extends Mach IPC across the network, so that tasks can exchange messages with remote tasks as easily as with local ones. The fundamental abstractions of Mach IPC are the message and the port. This section describes the data structures and functions that implement these abstractions.

6.5.1 Message Data Structures

A message is a collection of typed data. It can contain three basic types of data:

- Ordinary data that is not interpreted by the kernel and is passed on to the receiver by physically copying it.
- Out-of-line memory, used to transfer large chunks of data using *copy-on-write* techniques, as described in Section 6.7.2.

- Send or receive rights to ports.

The message is composed of a fixed-size header, immediately followed by a variable-size set of data components (Figure 6-7). The message header includes the following information:

- **type** — simple (ordinary data only) or complex (may have out-of-line memory or port rights).
- **size** of the entire message (including the header).
- **destination port**.
- **reply port** — send right to a port to which the receiver can send a reply; this field is set only when the sender wants a reply.
- **message ID** — may be used by applications as they desire.

The message is constructed in the sending task's address space before it is sent. At that time, the destination and reply ports are the task's local names for these ports. Before delivering the message, the kernel must transform these port names into values that are meaningful to the receiver. This is described in Section 0.

Each data component consists of a type descriptor, followed by the data itself. The descriptor contains the following information:

- **name** — identifies the type of data. Mach 2.5 recognizes 16 different name values, including *internal memory*, *port send or receive rights*, and scalars such as *byte*, *16-bit integer*,

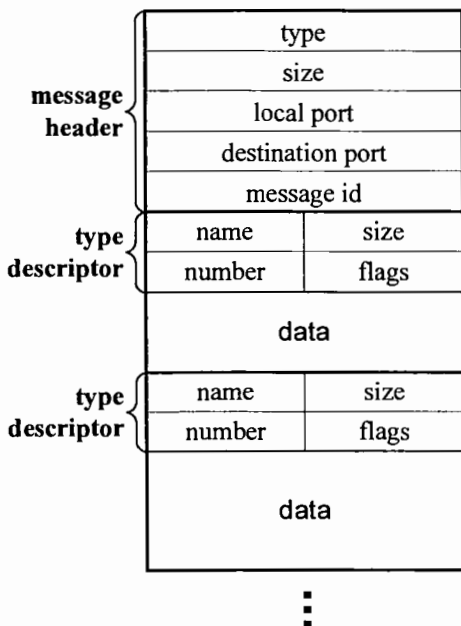


Figure 6-7. A Mach message.

32-bit integer, string, or real.

- **size** — size of each data item in this component.
- **number** — number of data items in this component.
- **flags** — specify if the data is in-line or out-of-line and if the memory or port rights must be deallocated from the sender task.

6.5.2 Message Passing Interface

Applications may use message passing in several different ways:

- To send a message, but not expect a reply.
- To wait for unsolicited messages and process them when they arrive.
- To send a message and expect a reply, but not wait for it. The application receives the reply asynchronously and processes it at a later, more convenient time.
- To send a message and wait for a reply.

The programming interface [Baro 90] to message passing consists of three functions that jointly allow all these forms of communication:

```
msg_send (msg_header_t* hdr,
          msg_option_t option,
          msg_timeout_t timeout);

msg_rcv  (msg_header_t* hdr,
          msg_option_t option,
          msg_timeout_t timeout);

msg_rpc  (msg_header_t* hdr,
          msg_size_t rcv_size,
          msg_option_t option,
          msg_timeout_t send_timeout,
          msg_timeout_t receive_timeout);
```

The *msg_send* call sends a message but does not expect a reply. The call may block if the destination port's message queue is full. Likewise, *msg_rcv* blocks until a message is received. Each call accepts a SEND_TIMEOUT or RCV_TIMEOUT option; if specified, the call blocks for a maximum of timeout milliseconds. After the timeout period expires, the call returns with a *timed out* status instead of remaining blocked. The RCV_NO_SENDERS option causes *msg_rcv* to return if no one else has a send right to the port.

The *msg_rpc* call sends an outgoing message, then waits for a reply to arrive. It is merely an optimized way of performing a *msg_send* followed by a *msg_rcv*. The reply reuses the message buffer used by the outgoing message. The options for *msg_rpc* include all the options of *msg_send* and *msg_rcv*.

The header contains the size of the message. When calling *msg_rcv*, the header contains the maximum size of the incoming message that the caller can accept; upon return, the header contains

the actual size received. In the *msg_rpc* call, *rcv_size* must be specified separately, because the header contains the size of the outgoing message.

6.6 Ports

Ports are protected queues of messages. Tasks can acquire send or receive rights or capabilities to a port. Ports can only be accessed by holders of the appropriate rights. Although many tasks may have send rights to a port, only one task can hold a receive right. The holder of a receive right automatically has a send right to that port.

Ports are also used to represent Mach objects such as tasks, threads, and processors. The kernel holds the receive rights to such ports. Ports are reference-counted, and each send right constitutes a reference to the object represented by the port. Such a reference allows the holder to manipulate the underlying object. For instance, the *task_self* port of a task represents that task. The task can send messages to that port to request kernel services that affect the task. If another task, perhaps a debugger, also has send rights to this port, it can perform operations on this task, such as suspending it, by sending messages to the port. The specific operations that are permitted depend on the object and the interface it wishes to export.

This section describes the port name space and the data structures used to represent a port.

6.6.1 The Port Name Space

Each capability or port right represents a name of that port. The names are simple integers, and the name space is local to each task. Thus different tasks may have different names for the same port; conversely, the same name may represent different ports in different tasks. In that sense, port names are similar to UNIX file descriptors.

Each task can have at most one name for any port. Port rights can be relayed through messages, and thus a task may acquire a right to the same port several times. The kernel ensures that the same name is reused each time. As a result, a task may compare two port names—if they do not match, they cannot refer to the same port.

Each port is also represented by a global kernel data structure. The kernel must translate the local port name to a global name (address of the global data structure for that port), and vice-versa. In the case of file descriptors, each UNIX process maintains a descriptor table in its *u* area that stores pointers to the corresponding open file objects. Mach uses a different translation method, described in Section 6.6.3.

6.6.2 The Port Data Structure

The kernel maintains a *kern_port_t* data structure for each port. It contains the following information:

- Reference count of all names (rights) to this port.
- Pointer to the task that holds the receive rights.
- Local name for the port in the receiver task.

- Pointer to a backup port. If this port is deallocated, the backup port receives all messages sent to it.
- Doubly linked list of messages.
- Queue of blocked senders.
- Queue of blocked receivers. Although a single task has receive rights, many threads in the task may be waiting to receive a message.
- Linked list of all translations for this object.
- Pointer to the *port set*, and pointers to next and previous ports in this set, if the port belongs to a port set (Section 6.8.3).
- Count of messages currently in the queue.
- Maximum number of messages (backlog) allowed in the queue.

6.6.3 Port Translations

Mach maintains one translation entry for each port right. The translation entries must manage a set of $\langle \text{task}, \text{port}, \text{local_name}, \text{type} \rangle$ tuples, where *task* is the task that owns the right, *port* is a pointer to the kernel data structure for the port, *local_name* is the name of the port within the task, and *type* is send or receive. Mach uses the translations in several different ways:

- *msg_send* must convert a $\langle \text{task}, \text{local_name} \rangle$ to *port*.
- *msg_rcv* must convert a $\langle \text{task}, \text{port} \rangle$ to *local_name*.
- When a task deallocates a port, it must find all rights for that port.
- When a task is destroyed, the kernel must find all port translations for that task and release the corresponding references.
- When a port is destroyed, the kernel must find all translations for that port, and notify tasks that hold rights to it.

This requires a translation scheme that can efficiently support all the above operations. Figure 6-8 describes the Mach 2.5 port translation data structures. Mach uses two global hash tables to find the entries quickly—TP_table hashes the entries based on $\langle \text{task}, \text{port} \rangle$, and TL_table hashes them based on $\langle \text{task}, \text{local_name} \rangle$. The kernel_port_t and task data structures hold the heads of the linked lists of translations for that port or task respectively.

Hence in Figure 6-8, the entries *a*, *b*, and *c* describe translations of different ports in the same task, while entries *c*, *d*, and *e* are translations of the same port in different tasks. Entries *b*, *d*, and *f* hash to the same index in the TP_table, while entries *e* and *g* hash to the same index in the TL_table. Each translation entry is described by a port_hash_t structure, which contains the following information:

- *task* — the task that owns this right.
- *local_name* — name for the right in this task.
- *type* — send or receive.
- *obj* — pointer to port object in kernel.

Additionally, each translation entry is on each of the following doubly linked lists:

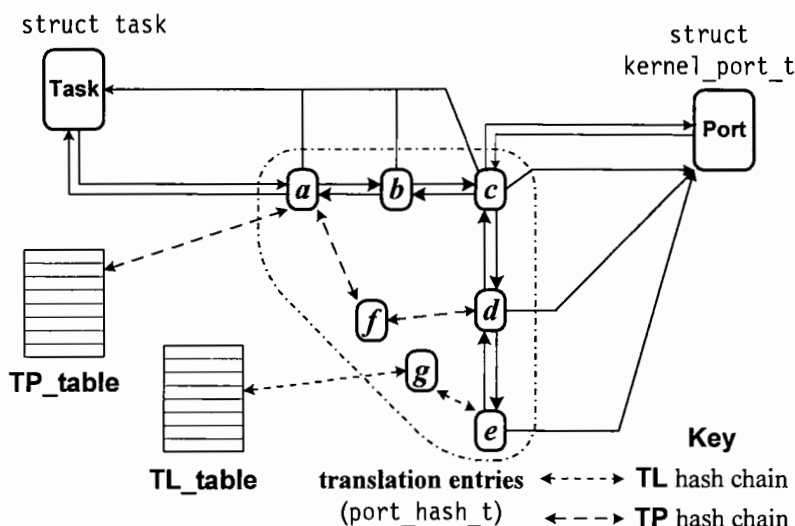


Figure 6-8. Port translation in Mach.

- TP_chain — hash chain based on <task, port>.
- TL_chain — hash chain based on <task, local_name>.
- task_chain — list of all translations owned by the same task.
- obj_chain — list of all translations for this port.

6.7 Message Passing

A single message transfer requires several operations:

1. The sender creates the message in its own address space.
2. The sender issues the *msg_send* system call to send the message. The message header contains the destination port.
3. The kernel copies the message into an internal data structure (*kern_msg_t*) using the *msg_copyin()* routine. In this process, port rights are converted to pointers to the ports' kernel objects, and out-of-line memory is copied into a holding map.
4. (a) If a thread is waiting to receive the message (the thread is on the blocked receivers queue of this port), it is awakened and the message is given to it directly.
 (b) Otherwise, if the port's message queue is full, the sender blocks until a message is removed from the queue.
 (c) Otherwise, the message is queued at the port, where it remains until a thread in the receiver task performs a *msg_rcv*.

5. The kernel returns from *msg_send* once the message has been queued or given to the receiver.
6. When the receiver calls *msg_rcv*, the kernel calls *msg_dequeue()* to remove a message from the queue. If the queue is empty, the receiver blocks until a message arrives.
7. The kernel copies the message into the receiver's address space using the *msg_copyout()* function, which performs further translations on out-of-line memory and port rights.
8. Often the sender expects a reply. For this to happen, the receiver must have a send right to a port owned by the sender. The sender sends this right to the receiver using the *reply port* field in the message header. In such a case, the sender would normally use the *mach_rpc* call to optimize this exchange. This call is semantically equivalent to a *msg_send* followed by a *msg_rcv*.

Figure 6-9 describes the transformations on the different components of a message during the transfer process. Let us now take a closer look at some important issues of message passing.

6.7.1 Transferring Port Rights

There are several reasons for transferring port rights in messages. The most frequent case is that of a reply port (Figure 6-10). A thread in task **T1** sends a message to port **P2**, which is owned by task **T2**. In this message, **T1** passes the send right to port **P1**, to which **T1** has the receive rights. As a result, **T2** can send the reply message to **P1**, and the sending thread will wait for it. This situation is so common that the message header itself contains a field to hold the reply port right.

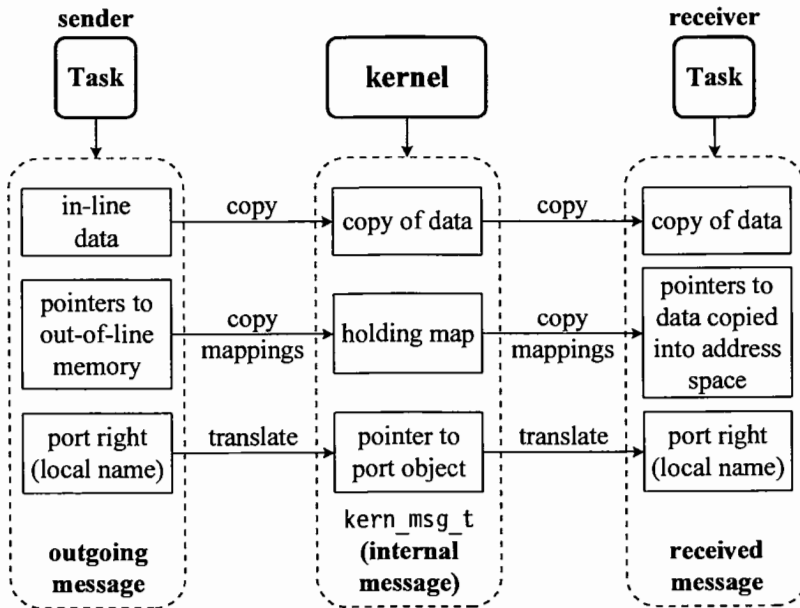


Figure 6-9. Two stages of a message transfer.

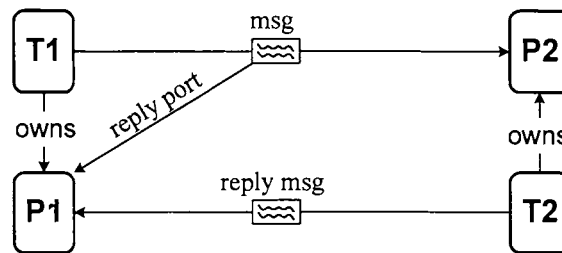


Figure 6-10. Messages can contain the send rights to a reply port.

Another common situation involves the interactions between a server program, a client, and a name server (Figure 6-11). The name server holds send rights to several server programs in the system. Typically, the servers register themselves with the name server when they begin executing (a). All tasks inherit a send right to a name server during task creation (this value is stored in the bootstrap port field in the task structure).

When a client wishes to access a server program, it must first acquire send rights to a port owned by the server. To do so, it queries the name server (b), which returns a send right to that server (c). The client uses that right to send a request to the server (d). The request contains a reply port that the server can use to reply to the client (e). Further interactions between this server-client pair need not involve the name server.

The sender sends a port right using its local name for the port. The type descriptor for that component of the message informs the kernel that the item is a port right. The local name means nothing to the receiver, and hence the kernel must translate it. To do this, the kernel searches the translation entries by hashing on `<task, local_name>` and identifies the kernel object (global name) for that port.

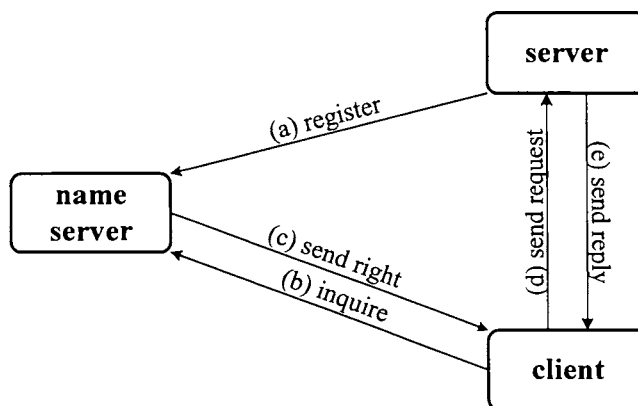


Figure 6-11. Using a name server to initiate contact between a client and a server.

When the message is retrieved, the kernel must translate this global name to a local name in the receiving task. It first checks if the receiver already has a right for this port (by hashing on *<task, port>*). If so, the kernel translates it to the same name. Otherwise, the kernel allocates a new port name in the receiver and creates a translation entry that maps the name to the port. Port names are usually small integers, and the kernel uses the lowest available integer for the new name.

Because the kernel creates an extra reference for this port, it must increment the reference count in the port object. The kernel does so when it copies the message into system space, since the new reference is created at that point. Alternately, the sender could have specified the deallocate flag in the type descriptor. In that case, the kernel deallocates the right in the sender task and does not need to increment the port reference count.

6.7.2 Out-of-Line Memory

If a message contains a small amount of data, it can be transferred by physically copying the data, first into a kernel buffer, and later (when retrieved) into the receiving task's address space. This approach, however, is expensive for large data transfers. Because Mach allows a message to contain as much as an entire address space (up to four gigabytes on 32-bit architectures), it must provide a more efficient way of transferring such data.

Typically, with large transfers, most of the data may never be modified by either the sender or the receiver. In such a case there is no need to make another copy of the data. A page must be copied only if and when one of the two tasks attempts to modify it. Until then, both tasks share a single physical copy of the page. Mach implements this method by using the copy-on-write sharing mechanisms of Mach's virtual memory subsystem. Chapter 15 describes Mach memory management in detail. Here, we restrict the discussion to IPC-related issues.

Figure 6-12 describes the transfer of out-of-line memory. The sender specifies out-of-line memory using a flag in the *type descriptor*. The `msg_copyin()` routine (called from `msg_send`) modifies the sender task's mappings for these pages to be read-only and copy-on-write. It then creates a temporary "holding map" for these pages in the kernel, and also marks those entries as read-only and copy-on-write (Figure 6-12(a)). When the receiver calls `msg_rcv`, the `msg_copyout()` function allocates an address range in the receiving task and copies the entries from the holding map to the receiver's address map. It marks the new entries as read-only and copy-on-write, and deallocates the holding map (Figure 6-12(b)).

At this point, the sender and receiver share the pages as copy-on-write. When either task attempts to modify such a page, it incurs a page fault. The fault handler recognizes the situation, and resolves the fault by making a new copy of the page and changing the faulting task's mappings to point to it. It also changes protections so that both tasks can now write to their copy of the page (Figure 6-12(c)).

Note that out-of-line memory transfer involves two phases—first, the message is placed on the queue and the pages are in transit; later, when the message is retrieved, the pages are shared by both sender and receiver. The holding map handles the transition phase. It ensures that if the sender modifies the page before the receiver retrieves it, the kernel will create a new copy for the sender, and the receiver will continue to access the original copy.

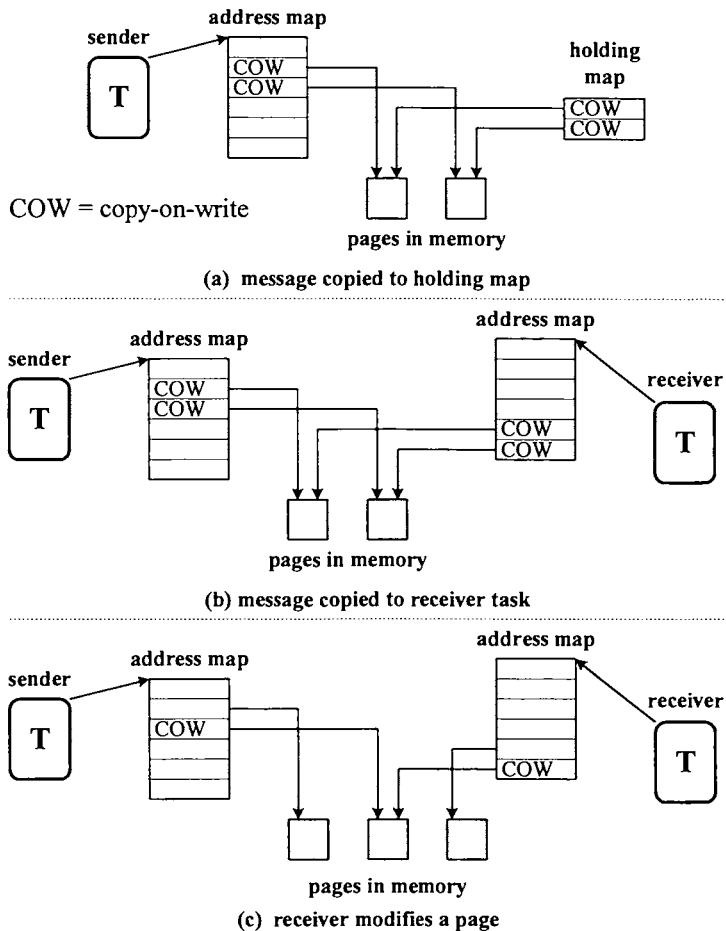


Figure 6-12. Transferring out-of-line memory.

This approach works best when neither the sender nor the receiver modifies the shared pages. This is true of many applications. Even if the pages are modified, this approach saves a copy operation. *In-line memory is copied twice—once from sender to kernel, then again from kernel to receiver. Out-of-line memory is copied at most once, the first time either task tries to modify it.*

The sender may set the deallocate flag in the type descriptor. In this case, the kernel does not use copy-on-write sharing. It simply copies the address map entries to the holding map during `msg_copyin()` and deallocates them from the sender's address map. When the message is retrieved, `msg_copyout()` copies the entries to the receiver's address map and deletes the holding map. As a result, the pages move from the sender's address space to that of the receiver without any data copying.

6.7.3 Control Flow

The message transfer proceeds along one of two paths—fast or slow. The slow path applies if a receiver is not waiting when a message is sent. In this case, the sender queues the message at the port and returns. When a receiver does a *msg_rcv*, the kernel dequeues the message and copies it into the receiver's address space.

Each port has a configurable limit, called its *backlog*, on the maximum number of messages that may be queued to it. When that limit is reached, the port is full and new senders will block until some messages are retrieved from the queue. Each time a message is retrieved from a port that has blocked senders, one sender will be awakened. When the last message is dequeued from the port, all blocked senders are awakened.

The fast path scenario occurs when a receiver is already waiting for the message. In this case, *msg_send* does not queue the message to the port. Instead, it wakes up the receiver and directly hands the message to it. Mach provides a facility called *handoff scheduling* [Drav 91], where one thread directly yields the processor to another specific thread. The fast path code uses this facility to switch to the receiver thread, which completes its *msg_rcv* call, using *msg_copyout()* to copy the message to its address space routine. This eliminates the overhead of queuing and dequeuing the message, and also speeds up the context switch, because the new thread to run is directly selected.

6.7.4 Notifications

A notification is an asynchronous message sent by the kernel to inform a task of certain events. The kernel sends the message to the task's *notify port*. Mach IPC uses three types of notifications:

NOTIFY_PORT_DESTROYED	When a port is destroyed, this message is sent to the owner of its backup port (if any). Port destruction and backup ports are discussed in the next section.
NOTIFY_PORT_DELETED	When a port is destroyed, this message is sent to all tasks that hold send rights to the port.
NOTIFY_MSG_ACCEPTED	When sending a message to a port whose queue is full, the sender can request this notification (using a SEND_NOTIFY option) when a message is removed from the queue.

The last case requires some elaboration. When the SEND_NOTIFY option is used, the message is transferred even if the queue is full. The kernel returns a SEND_WILL_NOTIFY status, which asks the sender not to send more messages to the queue until it receives the NOTIFY_MSG_ACCEPTED notification. This allows senders to send messages without blocking.

6.8 Port Operations

This section describes several operations on ports.

6.8.1 Destroying a Port

A port is destroyed when its receive right is released, typically when its owner task terminates. When that happens, a `NOTIFY_PORT_DELETED` notification is sent to all tasks that have a send right to it. Any messages waiting in the port's queue are destroyed. Blocked senders and receivers are awakened, and receive a `SEND_INVALID_PORT` or `RCV_INVALID_PORT` error respectively.

The destruction can become complex because the queued messages may contain rights to other ports. If any of these are receive rights, the corresponding ports must also be destroyed. In fact, a malicious user can send the receive right to a port in a message to that very port. Such situations are rare, but when they do occur, they cause some deadlock and unbounded recursion problems that Mach 2.5 cannot adequately resolve.

6.8.2 Backup Ports

The `port_set_backup` call assigns a *backup port* to a port. If a port having a backup is destroyed, the kernel does not deallocate the port; instead, it transfers the receive rights to the backup port.

Figure 6-13 illustrates this process. Port **P1** has previously assigned port **P2** as its backup. When **P1** is destroyed (perhaps due to termination of its owner task **T1**), the kernel sends a

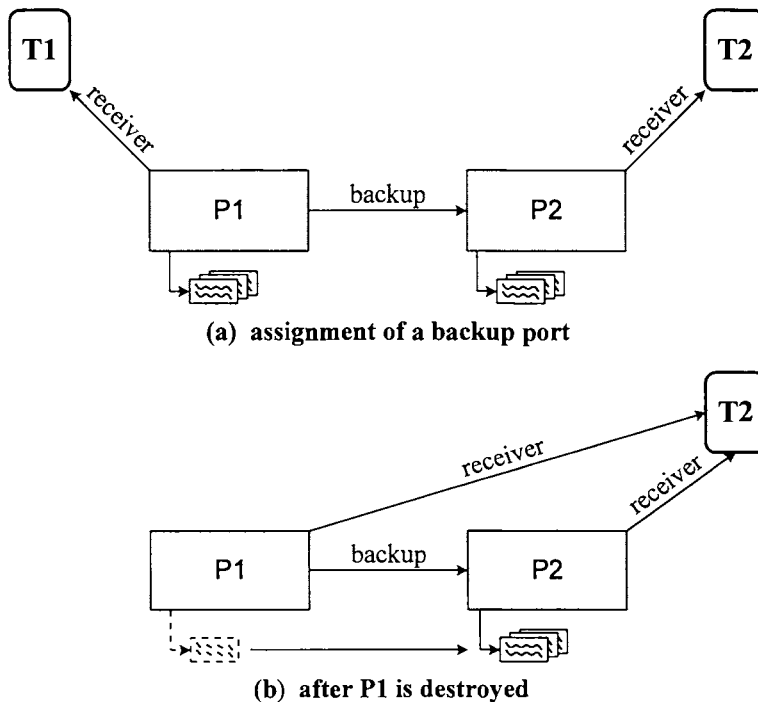


Figure 6-13. Backup port implementation.

NOTIFY_PORT_DESTROYED message to **P2**'s owner. **P1** is not deallocated, but holds send rights to **P2**. All messages sent to **P1** are automatically routed to **P2**, and can be retrieved by **P2**'s owner.

6.8.3 Port Sets

A port set (Figure 6-14) consists of a group of ports whose receive rights are replaced by a single set-wide receive right. Thus a single task can receive from the set. Moreover, it cannot receive selectively from individual ports in the set. In contrast, messages are sent to the constituent ports and not to the port set. Tasks have send rights to specific ports in the set, and when a message is received, it contains information about the port to which it was sent.

Port sets are useful when a server manages several objects. It may associate a port with each of them and place them all in a single port set. The server can then receive messages sent to any port in that set. Each message requests an operation on the port's underlying object. Because the message identifies the port it was sent to, the server knows which object to manipulate.

The port set functionality is comparable to that of the UNIX *select* system call, which allows a process to check for input on several descriptors. *An important difference is that the time taken to retrieve a message from a port set, or to send a message to a port in the set, is independent of the number of ports in the set.*

The port's kernel object contains a pointer to the set to which it belongs, as well as pointers to maintain a doubly linked list of all ports in a set. If the port is not a member of a port set, these pointers are NULL. The port set object contains a single message queue. The queues of the component ports are not used. Each port, however, retains its own count and limit (backlog) on the number of messages queued to it and also its own queue of blocked senders.

When a thread sends a message to a port that is a member of a set, the kernel checks that the port's backlog has not been exceeded (if so, it adds the sender to the port's blocked senders queue) and increments the count of messages queued to that port. It records the identity of the port in the message and places the message in the port set's queue. When a thread in the receiver task invokes a

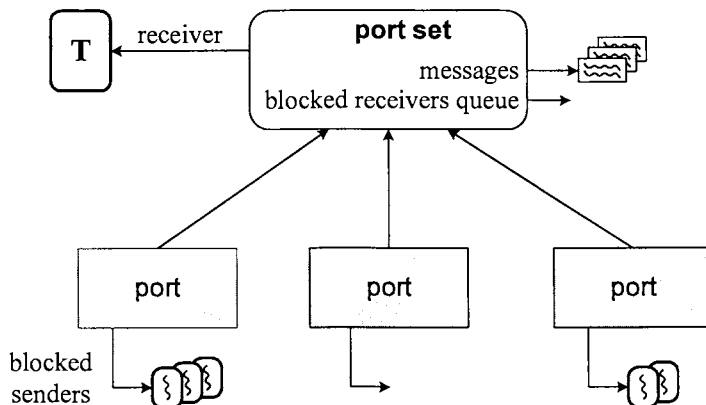


Figure 6-14. Port set implementation.

msg_rcv on a port set, the kernel retrieves the first message in the set's queue, regardless of the component port to which the message was queued.

Port sets are created and destroyed by the *port_set_allocate* and *port_set_deallocate* calls. Individual ports in the set are inserted and removed by the *port_set_insert* and *port_set_remove* calls.

6.8.4 Port Interpolation

Port interpolation allows a task to replace a right belonging to another task with a right to a different port, or to acquire a right from another task. This facility gives debuggers and emulators fine control over target tasks.

Figure 6-15 illustrates a possible scenario between a debugger task and a target task. The debugger first removes the target's send right to its *task_self* port using the *task_extract_send* call and obtains it for itself. It then calls *task_insert_send* to insert a send right to a port **P1**, owned by the debugger, in place of the target's *task_self* right. Similarly, the debugger uses the *task_extract_receive* and *task_insert_receive* calls to consume the target's receive right to its *task_notify* port, and substitute the receive right to another port **P2** to which the debugger has send rights.

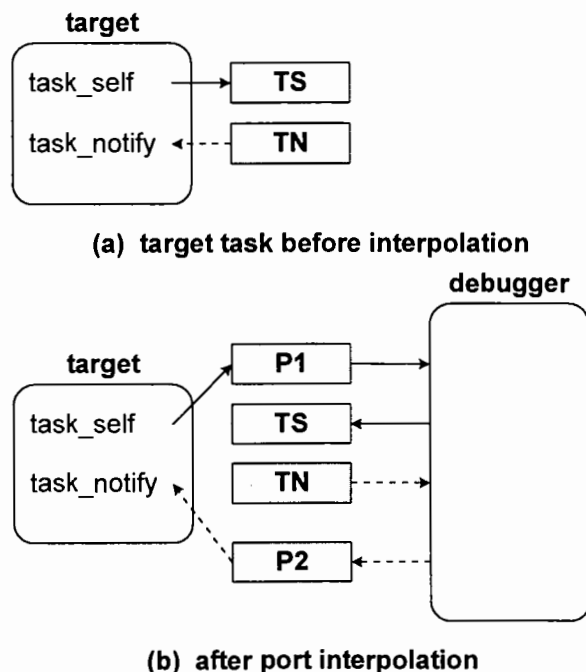


Figure 6-15. Port interpolation by a debugger.

Once this is accomplished, the debugger intercepts any messages sent by the target to its *task_notify* port (Mach system calls). The debugger processes the call and ensures that a reply is eventually sent to the reply port specified in the message. The debugger can choose to emulate the call and send the reply itself. Alternatively, it can forward the message to the kernel using the target's original *task_notify* port (to which the debugger now has send rights). When sending the message to the kernel, it can direct the reply to the target's reply port or specify its own reply port, thus intercepting the kernel's reply.

Likewise, the debugger intercepts any notifications sent to this task and decides whether to handle them on the target's behalf or forward them to the target. Hence a debugger (or other task) can control any port of a target task, provided it has send rights to the task's *task_self* port.

6.9 Extensibility

Mach IPC is designed to be transparently extensible to a distributed environment. A user-level program called the *netmsgserver* extends Mach IPC across a network, so that users can communicate with tasks on remote machines as easily as with local tasks. Applications are unaware of the remote connection and continue to use the same interface and system calls used for local communications. An application is typically unaware if it is communicating with a local or remote task.

There are two important reasons why Mach is able to provide such transparent extensibility. First, the port rights provide a location-independent name space. The sender simply sends the message using a local name of the port (the send right). It does not have to know if the port represents a local or remote object. The kernel maintains the mappings between the task's local name of the port and the port's kernel object.

Second, senders are anonymous. Messages do not identify the senders. The sender may pass the send right to a reply port in the message. The kernel translates this so that the receiver sees only a local name for this right, and cannot determine who the sender is. Moreover, the sender need not own the reply port; it only needs to own a send right to it. By specifying a reply port that is owned by another task, the sender can direct the reply to a different task. This is also useful for debugging, emulation, and so forth.

The *netmsgserver* operation is simple. Figure 6-16 shows a typical scenario. Each machine on the network runs a *netmsgserver* program. If a client on node **A** wishes to communicate with a server on node **B**, the *netmsgserver* on **A** sets up a proxy port to which the client sends the message. It then retrieves messages sent to that port and forwards them to the *netmsgserver* on machine **B**, which in turn forwards them to the server's port.

If the client expects a reply, it specifies a reply port in the message. The *netmsgserver* on **A** retains the send right to the reply port. The *netmsgserver* on **B** creates a proxy port for the reply port and sends the right to the proxy port to the server. The server replies to the proxy port, and the reply is routed via the two *netmsgservers* to the client's reply port.

Servers register themselves with the local *netmsgserver* and pass it the send right to a port to which the server listens. The *netmsgservers* maintain a distributed database of such registered network ports and provide them the same services (protection, notifications, etc.) that the kernel provides to local ports. Thus the *netmsgservers* query each other to provide a global name lookup

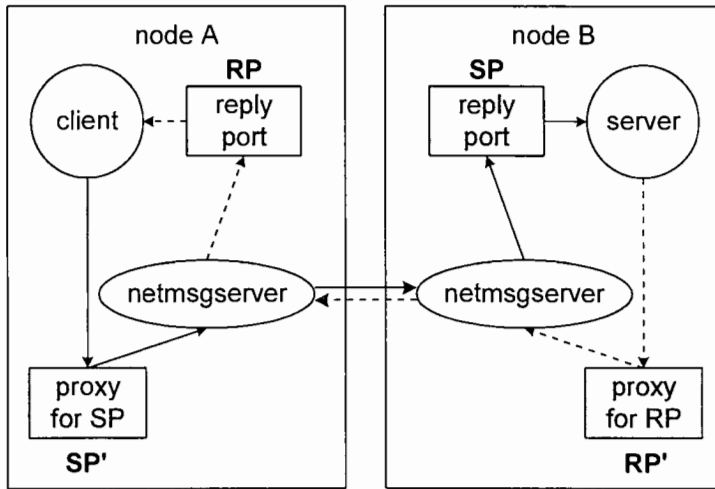


Figure 6-16. Remote communications using *netmsgservers*.

service. Tasks use this service to acquire send rights to ports registered with remote *netmsgservers*. In the absence of a network, this degenerates into a simple local name service.

The *netmsgservers* communicate with each other using low-level network protocols, not through IPC messages.

6.10 Mach 3.0 Enhancements

Mach 3.0 introduced several enhancements to the IPC subsystem [Drav 90], and addressed some important problems in the 2.5 implementation. The changes affect both the interface and the internal data structures and algorithms. This section explores some of the important changes.

A major problem in Mach 2.5 involves send rights. IPC is used heavily in client-server interactions. The client typically sends a message containing the send right to a reply port owned by the client. Once the server has sent the reply, it no longer needs to retain this right. It cannot, however, afford to deallocate the right. This is because another thread in the server application may have received, and may still be using, a send right to the same port, as the result of a separate message. The kernel translates the second right to the same local name, so that the server has a single send right to this port. If the first thread deallocates this right, the second thread will be unable to send its reply.

Servers work around this by never deallocating such send rights. This creates other problems. It is a security risk—the client may not want the server to hold this right indefinitely. It also unnecessarily uses up translation table entries, which impacts the entire system. Finally, it causes unnecessary notifications to be generated to servers when the client destroys this port. The kernel must send such notifications to all servers who have retained a send right to the port, and the servers must receive and process them, even though they have no real interest in this port.

Mach 3.0 has three separate enhancements that address this general problem—send-once rights, notification requests, and user-references for send rights.

6.10.1 Send-Once Rights

A *send-once right* to a port is, as its name implies, a send right that may only be used once. It is generated by the task that holds the port's receive right, but can then be transferred from task to task. It guarantees that a message will result from it. Normally, the send-once right is consumed by using it as a destination port in a (reply) message; it is destroyed when the message is received. If the right is destroyed in any other way, such as when the task that holds the right terminates abruptly, the kernel sends a send-once notification to the port instead.

The kernel maintains send-once rights separately from send rights. Thus if a task acquires a send right and a send-once right to the same port, it will have separate local names for the two rights. A task may acquire multiple send-once rights to the same port. Unlike send rights, each send-once right is given its own unique name. The reference count in the port's kernel object only reflects the outstanding send rights, and is not affected by send-once rights.

Mach 3.0 uses send-once rights to specify reply ports. This right is destroyed once the reply is received, so the server does not retain it indefinitely. This eliminates the unnecessary notifications when the client destroys the port.

6.10.2 Mach 3.0 Notifications

Mach 2.5 sends notifications asynchronously to the tasks in response to various system events. The tasks have no control over which notifications they receive. Usually, they have no interest in the event and simply discard the notification. Such unnecessary notifications degrade overall system performance.

Furthermore, a single task-wide notification port is too limiting. A thread could intercept and destroy a notification that another thread in the task may be awaiting. User-level libraries increase the likelihood of this scenario, because the main programs and the libraries each use notifications differently and independently.

Mach 3.0 sends notifications only to tasks that request them by explicitly calling *mach_port_request_notification*. This request specifies a send-once right for the notification port. Mach 3.0 thus allows several notification ports in a task. Each program component or thread can allocate its own notification port, and avoid contention with others.

6.10.3 User-Reference Counting of Send Rights

If a task acquires multiple send rights to a port, the kernel maps them all to a single local name in the task's port name space, thus combining them into a single send right. If a thread deallocates this right, no other thread will be able to use it, even if the other thread had acquired the right independently.

While send-once rights eliminate one source of this problem, they do not resolve it completely. Send-once rights are primarily used to provide reply ports to servers. Send rights may be

acquired in various ways. If a client wants to communicate with a server, the client acquires (through the name server) a send right to it. If multiple threads in the client independently initiate contact with the server, they will each receive a send right, which will be combined into a single name. If one of these threads deallocates the name, it will have an impact on all other threads.

Mach 3.0 addresses this problem by associating a user-reference with each send right. Thus in the previous example, the kernel increments the reference count each time the task obtains the same send right, and decrements it each time a thread deallocates the right. When the last reference is released, the kernel can remove the right safely.

6.11 Discussion

Mach uses IPC not only for communication between processes, but also as a fundamental kernel structuring primitive. The virtual memory subsystem uses IPC to implement copy-on-write [Youn 87], and the kernel uses IPC to control tasks and threads. The basic abstractions of Mach, such as *tasks*, *threads*, and *ports*, interact with one another through message passing.

This architecture provides some interesting functionality. For instance, the *netmsgserver* transparently extends the IPC mechanisms to a distributed system, so that a task may control and interact with objects on remote nodes. This allows Mach to provide facilities such as remote debugging, distributed shared memory, and other client-server programs.

In contrast, the extensive use of message passing results in poor performance. For a while there was a great interest in building microkernel operating systems, where most of the facilities are provided by user-level server tasks that communicate with one another using IPC. While many vendors are still working on such solutions, the performance concerns have driven these efforts away from the mainstream.

The proponents of Mach have argued that IPC performance is not an important factor in designing microkernel operating systems [Bers 92] because of the following reasons:

- Improvements in IPC performance have been far greater than in other areas of the operating system.
- With the increasing reliance on hardware caches, the cost of operating system services will be dominated by cache hit patterns. Since the IPC code is well localized, it can be easily tuned to use the cache optimally.
- Some data transfer can be achieved through other mechanisms such as shared memory.
- Migrating some of the kernel functionality into user-level servers reduces the number of mode switches and protection boundary crossings, which are expensive.

Researchers have devoted considerable attention to improving IPC performance [Bers 90, Barr 91]. So far, however, Mach IPC has had a limited impact in the commercial world. Even Digital UNIX, which is based on Mach, does not use Mach IPC in many of its kernel subsystems.

6.12 Summary

This chapter described several IPC mechanisms. Signals, pipes, and *ptrace* are universal facilities, available in all but the earliest UNIX systems. The System V IPC suite, comprising shared memory, semaphores, and message queues, is also available in most modern variants. In the Mach kernel, all objects use IPC to interact with each other. Mach IPC is extensible through the *netmsgserver*, allowing the development of distributed, client-server applications.

Some other IPC mechanisms are covered elsewhere in the book—file locking in Chapter 8, memory-mapped files in Chapter 14, and STREAMS pipes in Chapter 17.

6.13 Exercises

1. What are the limitations of *ptrace* as a tool for writing debuggers?
2. The *pid* argument of *ptrace* must specify the process ID of a child of the caller. What are the implications of relaxing this requirement? Why should processes not be able to use *ptrace* to interact with arbitrary processes?
3. Compare the IPC functionality provided by pipes and message queues. What are the advantages and drawbacks of each? When is one more suitable than the other?
4. Most UNIX systems allow a process to attach the same shared memory region to more than one location in its address space. Is this a bug or a feature? When would this be useful? What problems could it cause?
5. What issues must a programmer be concerned with in choosing an address to attach a shared memory region to? What errors would the operating system protect against?
6. How can the *IPC_NOWAIT* flag be used to prevent deadlocks when using semaphores?
7. Write programs to allow cooperating processes to lock a resource for exclusive use, using (a) FIFO files, (b) semaphores, (c) the *mkdir* system call, and (d) *flock* or *lockf* system calls. Compare and explain their performance.
8. What side effects, if any, must the programmer be concerned with in each of the above cases?
9. Is it possible to implement resource locking through (a) signals alone or (b) shared memory and signals? What would be the performance of such a facility?
10. Write programs to transfer a large amount of data between two processes, using (a) a pipe, (b) a FIFO, (c) a message queue, and (d) shared memory with semaphores for synchronization. Compare and explain their performance.
11. What are the security problems associated with System V IPC? How can a malicious program eavesdrop on, or interfere with, communications between other processes?
12. Semaphores are created with *semget* but initialized with *semctl*. Hence creation and initialization cannot be accomplished in a single atomic operation. Describe a situation where this might lead to a race condition and suggest a solution to the problem.
13. Can System V message queues be implemented on top of Mach IPC? What problems must such an implementation solve?

14. Why are send-once rights useful?
15. How do port sets help in developing a client-server application for Mach?

6.14 References

- [Bach 86] Bach, M.J., *The Design of the UNIX Operating System*, Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [Baro 90] Baron, R.V., Black, D., Bolosky, W., Chew, J., Draves, R.P., Golub, D.B., Rashid, R.F., Tevanian, A., Jr., and Young, M.W., *Mach Kernel Interface Manual*, Department of Computer Science, Carnegie-Mellon University, Jan. 1990.
- [Barr 91] Barrera, J.S., III, "A Fast Mach Network IPC Implementation," *Proceedings of the USENIX Mach Symposium*, Nov. 1991, pp. 1–12.
- [Bers 90] Bershad, B.N., Anderson, T.E., Lazowska, E.D., and Levy, H.M., "Lightweight Remote Procedure Call," *ACM Transactions on Computer Systems*, Vol. 8, No. 1, Feb. 1990, pp. 37–55.
- [Bers 92] Bershad, B.N., "The Increasing Irrelevance of IPC Performance for Microkernel-Based Operating Systems," *USENIX Workshop on Micro-Kernels and Other Kernel Architectures*, Apr. 1992, pp. 205–212.
- [Dijk 65] Dijkstra, E.W., "Solution of a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 8, Sep. 1965, pp. 569–578.
- [Drav 90] Draves, R.P., "A Revised IPC Interface," *Proceedings of the First Mach USENIX Workshop*, Oct. 1990, pp. 101–121.
- [Drav 91] Draves, R.P., Bershad, B.N., Rashid, R.F., and Dean, R.W., "Using Continuations to Implement Thread Management and Communication in Operating Systems," Technical Report CMU-CS-91-115R, School of Computer Science, Carnegie-Mellon University, Oct. 1991.
- [Faul 91] Faulkner, R. and Gomes, R., "The Process File System and Process Model in UNIX System V," *Proceedings of the 1991 Winter USENIX Conference*, Jan. 1991, pp. 243–252.
- [Pres 90] Presotto, D.L., and Ritchie, D.M., "Interprocess Communications in the Ninth Edition UNIX System," *UNIX Research System Papers*, Tenth Edition, Vol. II, Saunders College Publishing, 1990, pp. 523–530.
- [Rash 86] Rashid, R.F., "Threads of a New System," *UNIX Review*, Aug. 1986, pp. 37–49.
- [Salu 94] Salus, P.H., *A Quarter Century of UNIX*, Addison-Wesley, Reading, MA, 1994.
- [Stev 90] Stevens, R.W., *UNIX Network Programming*, Prentice-Hall, Englewood Cliffs, NJ, 1990.
- [Thom 78] Thompson, K., "UNIX Implementation," *The Bell System Technical Journal*, Vol. 57, No. 6, Part 2, Jul.-Aug. 1978, pp. 1931–1946.
- [Youn 87] Young, M., Tevanian, A., Rashid, R.F., Golub, D., Eppinger, J., Chew, J., Bolosky, W., Black, D., and Baron, R., "The Duality of Memory and Communication in the Implementation of a Multiprocessor Operating System," *Proceedings of the Eleventh ACM Symposium on Operating Systems Principles*, Nov. 1987, pp. 63–76.